

ch03 선형모델 - linear model

학습 목표

- 선형 모델(Linear Regression)에 대해 이해합니다.
- Diabetes 데이터 셋을 활용하여 회귀 모델을 만들어 봅니다.

학습 내용

- Diabetes 데이터 셋 불러오기

목차

- 01 데이터 불러오기
- 02 모델 생성 및 예측, 평가
- 03 회귀의 평가지표 알아보기
- 04 회귀의 평가지표 구하기

01 데이터 불러오기

목차로 이동하기

```
In [3]: from sklearn.datasets import load_diabetes
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# 1. 데이터 불러오기
diabetes = load_diabetes()

X = diabetes.data
y = diabetes.target

# 2. DataFrame 변환
df = pd.DataFrame(X, columns=diabetes.feature_names)
df['target'] = y
display(df.head())
```

	age	sex	bmi	bp	s1	s2	s3	s4
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592

features	내용	값(범위 예시)
age	나이(표준화된 값)	--
sex	성별(이진 값을 표준화한 값)	--
bmi	체질량지수 BMI(표준화된 값)	--
bp	평균 혈압(표준화된 값)	--
s1	혈청 지표 1: 총 콜레스테롤(TCH) (표준화)	--
s2	혈청 지표 2: LDL 콜레스테롤(LDL) (표준화)	--
s3	혈청 지표 3: HDL 콜레스테롤(HDL) (표준화)	--
s4	혈청 지표 4: TCH/HDL 비율(표준화)	--
s5	혈청 지표 5: 중성지방 로그값(LTG) (표준화)	--
s6	혈청 지표 6: 혈당(GLU) (표준화)	--
(target) target	1년 뒤 당뇨 진행 정도(회귀 타겟)	--

In [4]: `df.shape, df.columns`

Out[4]: ((442, 11),
Index(['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6',
 'target'],
 dtype='object'))

02 모델 생성 및 예측, 평가

목차로 이동하기

```
In [5]: # 3. 모델 생성
        model = LinearRegression()

        # 4. 학습
        model.fit(X, y)

        # 5. 예측
        y_pred = model.predict(X)

        # 6. 평가
```

```
print("MSE:", mean_squared_error(y, y_pred))
print("R2 Score:", r2_score(y, y_pred))
```

MSE: 2859.6963475867506

R2 Score: 0.5177484222203498

데이터 나누기 - 학습, 평가

```
In [6]: X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                         test_size=0.3,
                                                         random_state=42)
```

```
In [7]: model = LinearRegression().fit(X_train, y_train) # 학습
pred = model.predict(X_test)
pred
```

```
Out[7]: array([138.46970031, 181.10052342, 125.34400904, 292.75977277,
               123.8830531 , 91.89830434, 257.26463123, 177.76169318,
               84.98549706, 109.15960992, 94.4855284 , 166.13235108,
               57.40742502, 206.13897354, 97.7811842 , 131.00472765,
               214.29789972, 252.52907661, 199.66656381, 219.49985634,
               203.23671317, 88.00656925, 70.65108459, 186.92233427,
               155.81266751, 162.81022205, 191.93135706, 184.72924276,
               46.62920829, 108.26566599, 178.14743952, 91.35065005,
               127.72125745, 184.04205666, 172.23799897, 189.51548863,
               121.83265708, 117.75339206, 145.67950306, 58.57563401,
               72.55421321, 107.27571105, 166.15280313, 153.29339984,
               165.19282154, 62.95752038, 73.58909449, 110.05656189,
               54.49723354, 164.88920632, 152.49472803, 63.8874565 ,
               111.4354561 , 108.34936269, 179.96973311, 158.70054112,
               95.04833555, 209.68990406, 118.37356519, 69.69946093,
               189.04680627, 204.99138626, 140.26840176, 105.75963198,
               131.22316354, 201.65649349, 173.60251683, 164.25810664,
               122.01567481, 143.53425629, 179.42286037, 201.32086534,
               234.50804093, 146.96093984, 80.27567403, 160.86822173,
               192.72110552, 208.6585054 , 158.56823462, 206.00933474,
               111.53729194, 140.28763033, 51.15323818, 54.76434639,
               113.02207773, 78.46425825, 80.20416459, 56.97774667,
               168.51207071, 185.55749267, 152.33436369, 242.66080008,
               106.44250735, 61.34968205, 53.44975218, 197.78538801,
               246.62420102, 185.25237675, 105.80114548, 62.42547543,
               193.39002547, 107.08821595, 297.08578592, 94.45306615,
               148.41869832, 104.10666888, 128.66940497, 126.90012169,
               165.84858396, 186.14546255, 108.5888133 , 203.83170771,
               204.97622269, 209.10156998, 191.65509488, 161.26918179,
               148.51075431, 135.0455994 , 111.74197539, 163.62966339,
               170.55301782, 95.53608319, 46.85961143, 164.34533761,
               127.1747026 , 176.61035076, 173.47305611, 107.40153597,
               202.99778708, 92.10100436, 122.28876806, 80.80866991,
               233.21870492])
```

예측값이 0이하의 값이 있다. 이 경우 RMLSE에서 에러 발생.
pred를 0이하는 0으로 처리한다.

```
In [8]: type(pred)
```

```
Out[8]: numpy.ndarray
```

```
In [9]: pred[pred < 0] = 0
pred
```

```
Out[9]: array([138.46970031, 181.10052342, 125.34400904, 292.75977277,
123.8830531 , 91.89830434, 257.26463123, 177.76169318,
84.98549706, 109.15960992, 94.4855284 , 166.13235108,
57.40742502, 206.13897354, 97.7811842 , 131.00472765,
214.29789972, 252.52907661, 199.66656381, 219.49985634,
203.23671317, 88.00656925, 70.65108459, 186.92233427,
155.81266751, 162.81022205, 191.93135706, 184.72924276,
46.62920829, 108.26566599, 178.14743952, 91.35065005,
127.72125745, 184.04205666, 172.23799897, 189.51548863,
121.83265708, 117.75339206, 145.67950306, 58.57563401,
72.55421321, 107.27571105, 166.15280313, 153.29339984,
165.19282154, 62.95752038, 73.58909449, 110.05656189,
54.49723354, 164.88920632, 152.49472803, 63.8874565 ,
111.4354561 , 108.34936269, 179.96973311, 158.70054112,
95.04833555, 209.68990406, 118.37356519, 69.69946093,
189.04680627, 204.99138626, 140.26840176, 105.75963198,
131.22316354, 201.65649349, 173.60251683, 164.25810664,
122.01567481, 143.53425629, 179.42286037, 201.32086534,
234.50804093, 146.96093984, 80.27567403, 160.86822173,
192.72110552, 208.6585054 , 158.56823462, 206.00933474,
111.53729194, 140.28763033, 51.15323818, 54.76434639,
113.02207773, 78.46425825, 80.20416459, 56.97774667,
168.51207071, 185.55749267, 152.33436369, 242.66080008,
106.44250735, 61.34968205, 53.44975218, 197.78538801,
246.62420102, 185.25237675, 105.80114548, 62.42547543,
193.39002547, 107.08821595, 297.08578592, 94.45306615,
148.41869832, 104.10666888, 128.66940497, 126.90012169,
165.84858396, 186.14546255, 108.5888133 , 203.83170771,
204.97622269, 209.10156998, 191.65509488, 161.26918179,
148.51075431, 135.0455994 , 111.74197539, 163.62966339,
170.55301782, 95.53608319, 46.85961143, 164.34533761,
127.1747026 , 176.61035076, 173.47305611, 107.40153597,
202.99778708, 92.10100436, 122.28876806, 80.80866991,
233.21870492])
```

```
In [6]: from IPython.display import display, Image
import mglearn
display(Image(filename='img/linear_model01.png'))
```

$$\hat{y} = w_1 * x_1 + w_2 * x_2 + \dots + w_p * x_p + b$$

$$\hat{y} = w_1 * x_1 + b$$

는 특성(feature) 하나를 선택한 모델 ``

- $x_1 \sim x_p$ 는 데이터 포인트에 대한 특성(feature)
- w 와 b 는 모델이 학습할 파라미터
- 선형회귀 또는 최소제곱법(OLS)은 가장 간단하고 오래된 회귀용 선형 알고리즘.

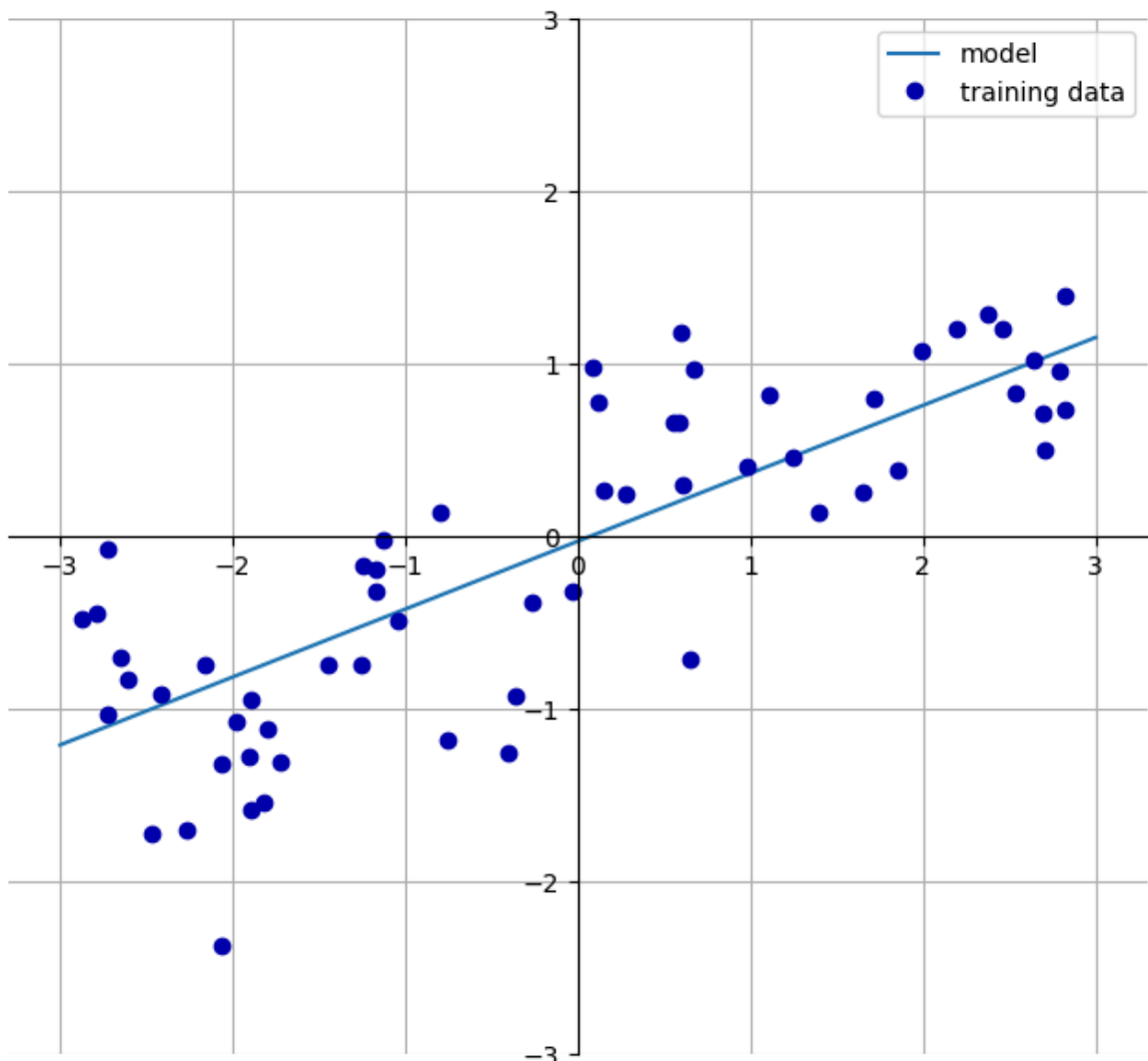
- 선형 회귀는 예측과 훈련 세트에 있는 타겟 y 사이의 평균제곱오차(mean squared error)를 최소화하는 파라미터 w 와 b 를 찾는다.
- 평균 제곱 오차는 예측값과 타겟값의 차이를 제곱하여 더한 후에 샘플의 개수로 나눈 것.

```
In [7]: display(Image(filename='img/linear_model02_mse.png'))
```

$$\text{MSE(평균제곱오차)} = \frac{\sum (y_i - \hat{y}_i)^2}{n}$$

```
In [8]: mglearn.plots.plot_linear_regression_wave()
```

w[0]: 0.393906 b: -0.031804



03 회귀의 평가지표 알아보기

[목차로 이동하기](#)

```
In [10]: import pandas as pd
import numpy as np
```

```
In [11]: error = y_test - pred
error2 = (y_test - pred)**2

dict_dat = {"실제값":y_test, "예측값":pred, "오차":error,
            "오차제곱":error2}
dat = pd.DataFrame(dict_dat )
dat
```

```
Out[11]:
```

	실제값	예측값	오차	오차제곱
0	219.0	138.469700	80.530300	6485.129167
1	70.0	181.100523	-111.100523	12343.326305
2	202.0	125.344009	76.655991	5876.140950
3	230.0	292.759773	-62.759773	3938.789078
4	111.0	123.883053	-12.883053	165.973057
...	...	...	...	...
128	257.0	202.997787	54.002213	2916.239001
129	137.0	92.101004	44.898996	2015.919810
130	63.0	122.288768	-59.288768	3515.158018
131	93.0	80.808670	12.191330	148.628529
132	232.0	233.218705	-1.218705	1.485242

133 rows × 4 columns

```
In [12]: dat['오차절대값'] = abs(dat['오차'])
dat
```

Out[12]:

	실제값	예측값	오차	오차제곱	오차절대값
0	219.0	138.469700	80.530300	6485.129167	80.530300
1	70.0	181.100523	-111.100523	12343.326305	111.100523
2	202.0	125.344009	76.655991	5876.140950	76.655991
3	230.0	292.759773	-62.759773	3938.789078	62.759773
4	111.0	123.883053	-12.883053	165.973057	12.883053
...	...	...	...	...	...
128	257.0	202.997787	54.002213	2916.239001	54.002213
129	137.0	92.101004	44.898996	2015.919810	44.898996
130	63.0	122.288768	-59.288768	3515.158018	59.288768
131	93.0	80.808670	12.191330	148.628529	12.191330
132	232.0	233.218705	-1.218705	1.485242	1.218705

133 rows × 5 columns

평가 지표

- 모델을 평가하기 위해 회귀모델은 일반적으로 사용하는 지표는 다음을 사용합니다.
 - MAE(mean absolute error) : 평균 절대값 오차
 - MAPE(mean absolute percentage error) : 평균 절대값 백분율 오차
 - MSE(mean squared error) : 평균 제곱 오차
 - RMSE(root mean squared error) : 평균 제곱근 오차
 - RMLSE(Root Mean Squared Logarithmic Error)

```
In [13]: ### MAE (mean absolute error)
mae_val = dat['오차절대값'].sum() / dat.shape[0]
mae_val
```

Out[13]: np.float64(41.91937845679274)

```
In [14]: ### MSE (mean squared error)
mse_val = dat['오차제곱'].sum() / dat.shape[0]
mse_val
```

Out[14]: np.float64(2821.750981001311)

```
In [15]: ### RMSE(Root Mean Squared Error)
rmse_val = np.sqrt(mse_val)
rmse_val
```

Out[15]: np.float64(53.120156070942706)

(실습) RMLSE 값을 구해보자.

```
In [16]: dat['실제값_log'] = np.log( dat['실제값'] + 1 )
dat['예측값_log'] = np.log( dat['예측값'] + 1 )
dat['log오차제곱'] = (dat['실제값_log'] - dat['예측값_log']) ** 2
```

```
In [17]: dat
```

```
Out[17]:
```

	실제 값	예측값	오차	오차제곱	오차절대값	실제값 _log	예측값 _log	lo
0	219.0	138.469700	80.530300	6485.129167	80.530300	5.393628	4.937847	0.2
1	70.0	181.100523	-111.100523	12343.326305	111.100523	4.262680	5.204559	0.8
2	202.0	125.344009	76.655991	5876.140950	76.655991	5.313206	4.839008	0.2
3	230.0	292.759773	-62.759773	3938.789078	62.759773	5.442418	5.682762	0.0
4	111.0	123.883053	-12.883053	165.973057	12.883053	4.718499	4.827378	0.0
...	...	...	...	...	...	...	...	...
128	257.0	202.997787	54.002213	2916.239001	54.002213	5.552960	5.318109	0.0
129	137.0	92.101004	44.898996	2015.919810	44.898996	4.927254	4.533685	0.1
130	63.0	122.288768	-59.288768	3515.158018	59.288768	4.158883	4.814529	0.4
131	93.0	80.808670	12.191330	148.628529	12.191330	4.543295	4.404383	0.0
132	232.0	233.218705	-1.218705	1.485242	1.218705	5.451038	5.456255	0.0

133 rows × 8 columns



```
In [18]: ### RMLSE(Root Mean )
mlse_val = dat['log오차제곱'].sum() / dat.shape[0]
print("mlse value :", mlse_val)

rmlse_val = np.sqrt(mlse_val)
print("rmlse value :", rmlse_val)
```

mlse value : 0.1700892706108052

rmlse value : 0.4124188048704923

04 회귀의 평가지표 구하기

목차로 이동하기

sklearn을 활용한 평가지표 구하기

```
In [19]: import numpy as np

from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_squared_log_error
```



```
In [20]: ### MAE(mean absolute error) : 평균 절대값 오차  
mean_absolute_error(y_test, pred)
```

```
Out[20]: 41.91937845679274
```

```
In [21]: ### MSE (mean squared error)  
mean_squared_error(y_test, pred)
```

```
Out[21]: 2821.750981001311
```

```
In [22]: ### MSLE (mean squared log error)  
mean_squared_log_error(y_test, pred)
```

```
Out[22]: 0.1700892706108052
```

```
In [23]: ### RMSE(Root Mean Squared Error)  
rmse_val = np.sqrt( mean_squared_error(y_test, pred) )  
rmse_val
```

```
Out[23]: np.float64(53.120156070942706)
```

```
In [24]: ### RMLSE(Root Mean Log Squared Error)  
rmlse_val = np.sqrt( mean_squared_log_error(y_test, pred) )  
rmlse_val
```

```
Out[24]: np.float64(0.4124188048704923)
```

평가지표를 함수로 만들기

MAE (mean absolute error)

- 각각의 값에 절대값을 취한다. 이를 전부 더한 후, 갯수로 나누어주기

```
In [25]: def mae_1(y_test, y_pred):  
    error = y_test - y_pred  
    abs_error = np.abs(error)  
    mae_val = np.mean(abs_error)  
    return mae_val  
  
mae_1(y_test, pred)
```

```
Out[25]: np.float64(41.91937845679274)
```

MSE (mean squared error)

- 각각의 데이터의 (실제값-예측값) ^ 2 의 합을 데이터의 샘플의 개수로 나누어준것

```
In [26]: def mse_1(y_test, y_pred):  
    error = y_test - y_pred  
    error_2 = (error) ** 2  
    mse_val = np.mean(error_2)  
    return mse_val  
  
mse_1(y_test, pred)
```

Out[26]: np.float64(2821.750981001311)

RMSE (root mean squared error)

- 각 데이터의 (실제값-예측값) $\wedge 2$ 의 합을 데이터의 샘플의 개수로 나누어 준 이후에 제곱근 씌우기

```
In [27]: def rmse_1(y_test, y_pred):
          error = y_test - y_pred
          error_2 = (error) ** 2
          rmse_val = np.sqrt( np.mean(error_2) )
          return rmse_val

          rmse_1(y_test, pred)
```

Out[27]: np.float64(53.120156070942706)

MAPE(Mean Absolute Percentage Error)

- MAE를 퍼센트로 변환

```
In [28]: def MAPE(y_test, y_pred):
          error = (y_test - y_pred)/y_test
          mape_error = np.abs(error)
          return np.mean( mape_error ) * 100

          MAPE(y_test, pred)
```

Out[28]: np.float64(36.67196318312674)

MPE(Mean Percentage Error)

- MAPE에서 절대값을 제외한 지표

```
In [29]: def MAE(y_test, y_pred):
          return np.mean( (y_test - y_pred) / y_test ) * 100

          MAE(y_test, pred)
```

Out[29]: np.float64(-12.79090037504617)

RMLSE

```
In [30]: def rmsle(y_test, y_pred):
          log_y = np.log1p(y_test)
          log_pred = np.log1p(y_pred)
          squared_error = (log_y - log_pred) ** 2
          rmsle = np.sqrt(np.mean(squared_error))
          return rmsle

          rmsle(y_test, pred)
```

Out[30]: np.float64(0.4124188048704923)

- 만약, 값 중에 음수가 있을 경우, 예측값이 0이하의 값이 존재하므로 $\log_{10}()$ 해도 무한으로 음수에 가까워짐. 정상적인 동작이 안되므로 이를 처리(음수 제거) 해주어야 한다.

실습 과제

kaggle 데이터 셋 : <https://www.kaggle.com/datasets/harlfoxem/housesalesprediction>

- 실습 01. 데이터 나누기
- 실습 02. 선형회귀 모델 만들기
- 실습 03. MAE, MSE, RMSE 지표로 비교해 보기